TP1 - Data Engineering

Extracción de una API y almacenamiento en Delta Lake

Guía paso a paso, explicada desde cero, con la API del Art Institute of Chicago

Esta guía está pensada para que **entiendas** cada línea que escribís, no solo para que el programa funcione. Cada función, cada parámetro y cada librería se explica antes de usarse. Podés leerla de corrido o ir salteando a la parte que necesites.

Parte	Qué vas a encontrar
1. La consigna y el plan	Qué piden, qué endpoints elegimos y por qué.
2. Conceptos clave	API, JSON, DataFrame, Delta Lake, full vs incremental, watermark, merge.
3. Preparar el entorno	Qué se instala con pip y qué ya viene con Python.
4. La API del museo	Endpoints, parámetros, paginación y límites.
5. El header AIC-User-Agent	Qué es, por qué existe y cómo se usa.
6. Las librerías en detalle	time, pathlib, requests, pandas, deltalake y pyarrow: funciones y opciones.
7. Paso a paso (pasos 1 a 11)	El código completo, construido de a una pieza y explicado línea por línea.
8. Script completo	Todo junto, para copiar y pegar.
9. Cómo verificar	Comandos para comprobar que se guardó bien.
10. Errores comunes	Qué dice el error, por qué pasa y cómo se arregla.
11. Entrega	Checklist contra los criterios de evaluación y texto modelo para el comentario.

1. La consigna y el plan

La consigna del módulo pide un programa en Python que haga tres cosas, en este orden:

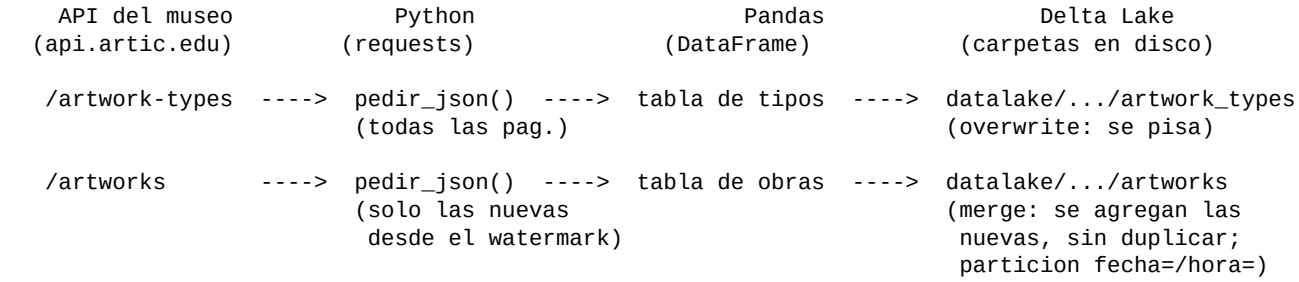
- **Extraer** datos de una API usando la librería `requests`, de **2 o más endpoints** de la misma API.
- **Convertir** lo obtenido en **DataFrames de Pandas**.
- **Guardar** esos DataFrames en **formato Delta Lake**, cada uno en su propio directorio, como en un data lake.

Además hay condiciones específicas que conviene tener a la vista:

Lo que pide la consigna	Cómo lo resolvemos
Un endpoint con datos **temporales** que se actualicen al menos una vez al día	`/artworks`: cada obra tiene un campo `updated_at` y el museo actualiza su catálogo todos los días.
Otro endpoint con datos **estáticos o metadatos**	`/artwork-types`: catálogo chico (unos 44 tipos de obra: Painting, Sculpture, Print...) que casi no cambia.
Extracción **incremental y full**, según corresponda	Obras: incremental (solo lo nuevo). Tipos de obra: full (todo, cada vez).
Cada DataFrame en su directorio de Delta Lake	`datalake/bronze/artic/artworks` y `datalake/bronze/artic/artwork_types`.
En incremental, **particionar por fecha y hora**	Columnas `fecha` y `hora` derivadas de `updated_at`, usadas en `partition_by`.
Los estáticos se pueden particionar o no	No los particionamos: son 44 filas.
Crear los directorios si no existen	`Path.mkdir(parents=True, exist_ok=True)`.
Pisar los datos o insertar nuevos según corresponda	Full: `mode="overwrite"` (pisa). Incremental: `merge` que inserta solo lo que no existe.
Datos crudos, sin transformaciones o con leves transformaciones	Solo agregamos `fecha` y `hora` y pasamos todo a texto. La limpieza real es del TP2.

> **OJO.** Esta API no es una serie de tiempo pura como el clima o las cotizaciones. El "dato temporal" acá es que las obras se actualizan a lo largo del tiempo. Antes de entregar, confirmá con el tutor que lo acepta. Si no, una alternativa dentro de la misma API es `/exhibitions` (muestras con fechas de inicio y fin).

El flujo completo en una imagen



2. Conceptos clave

Si es la primera vez que ves estos términos, leé esta sección con calma. Todo lo que sigue se apoya en ellos.

API y endpoint

Una **API** es una forma estructurada de que un programa le pida datos a otro. La del museo funciona por HTTP, igual que una página web, pero en vez de devolver una página con diseño devuelve datos puros. Cada "ventanilla" de esa API es un **endpoint**: una URL específica. Por ejemplo `https://api.artic.edu/api/v1/artworks` devuelve obras y `https://api.artic.edu/api/v1/artwork-types` devuelve tipos de obra.

JSON

Es el formato en el que responde la API: texto con una estructura de llaves y corchetes que se parece a un diccionario de Python. Ejemplo:

```
{"id": 1, "title": "Painting", "aat_id": 300033618}
```

Una lista de registros se ve como `[{...}, {...}, {...}]`. La librería `requests` lo convierte a diccionarios y listas de Python con `.json()`.

Paginación

La API no te da las más de 100.000 obras de golpe: las entrega por **páginas**. Vos elegís el tamaño (`limit`, máximo 100) y qué página querés (`page`). Cada respuesta trae un bloque `pagination` que te dice cuántas páginas hay en total. Para traer todo hay que ir pidiendo página 1, 2, 3... hasta llegar a la última.

DataFrame

Es la estructura de Pandas para tablas: filas y columnas con nombre, como una hoja de Excel dentro de Python. Una lista de diccionarios (el `data` que devuelve la API) se convierte directo en un DataFrame: cada diccionario es una fila y cada clave es una columna.

Parquet y Delta Lake

Parquet es un formato de archivo para tablas, comprimido y columnar, muy usado en data engineering. **Delta Lake** es una capa que se pone sobre archivos Parquet y agrega un registro de transacciones: una carpeta oculta `_delta_log` donde se anota cada cambio (qué archivos se agregaron, cuáles se quitaron). Gracias a eso podés:

- **Sobrescribir** una tabla de forma segura (`overwrite`).
- **Agregar** filas nuevas (`append`) o hacer **merge** (insertar solo las que no existen, o actualizar las que cambiaron).
- Consultar el **historial** de versiones de la tabla y volver a una versión anterior.
- Evitar que una tabla quede a medio escribir si algo falla en el medio: cada escritura es una transacción.

Físicamente, una tabla Delta es **una carpeta**: adentro hay archivos `.parquet` con los datos y la carpeta `_delta_log` con los metadatos. Cuando la consigna dice "guardar cada DataFrame en un directorio específico" se refiere a eso.

Data lake y capas (bronze)

Un **data lake** es un almacenamiento donde se guardan datos en su forma original o casi original, organizados en carpetas. Es común ordenarlo en capas: **bronze** (datos crudos tal como llegan), **silver** (limpios y procesados), **gold** (listos para análisis). Este TP es la capa bronze; la limpieza y el procesamiento vienen en la próxima entrega, como avisa la consigna.

Extracción full vs incremental

	Full	Incremental
Qué trae	Todo el contenido del endpoint, cada vez	Solo lo que cambió desde la última corrida
Cómo se guarda	Se pisa la tabla (`overwrite`)	Se agregan las filas nuevas (`merge` o `append`)
Cuándo conviene	Pocos datos o datos que casi no cambian	Muchos datos o que cambian seguido
En este TP	`/artwork-types` (unas 44 filas)	`/artworks` (más de 100.000 filas)

Watermark (marca de agua)

Es el valor que usás para saber "hasta dónde llegué la última vez". Acá es la fecha `updated_at` más reciente que ya está guardada en la tabla de obras. En cada corrida pedís solo lo que tenga un `updated_at` posterior. Lo bueno de calcularlo leyendo la propia tabla Delta es que no necesitás un archivo de estado aparte.

Partición

Es dividir una tabla en subcarpetas según el valor de una o más columnas. Si particionás por `fecha` y `hora`, Delta guarda los datos en carpetas como `fecha=2026-10-27/hora=14/`. Sirve para que las consultas por fecha lean solo las carpetas que necesitan.

Idempotencia y merge

Un proceso es **idempotente** si correrlo dos veces seguidas da el mismo resultado que correrlo una. Es una propiedad muy valorada: si el script se corta y lo volvés a lanzar, no queremos filas duplicadas. El `merge` de Delta lo logra: compara cada fila nueva con las existentes según una condición (el "predicado") y solo inserta las que no encuentra.

3. Preparar el entorno

Qué ya viene con Python y qué se instala con pip

Librería	¿Hay que instalarla?	Para qué la usamos
`time`	No, viene con Python	Esperar entre pedidos (`time.sleep`).
`pathlib`	No, viene con Python	Manejar rutas de carpetas (`Path`).
`requests`	Sí (`pip`)	Hacer los pedidos HTTP a la API.
`pandas`	Sí (`pip`)	Armar y manipular los DataFrames.
`deltalake`	Sí (`pip`)	Escribir y leer tablas Delta Lake sin necesitar Spark.
`pyarrow`	Sí (`pip`)	Motor que usa Pandas para pasarle los datos a `deltalake`.

Comandos de instalación

En la terminal (la de tu editor o la del sistema):

```
pip install requests pandas deltalake pyarrow
```

Si usás **Google Colab** o un notebook de Jupyter, se hace desde una celda y se le agrega un signo de exclamación adelante:

```
!pip install deltalake pyarrow
```

En Colab, `requests` y `pandas` ya vienen instalados. El tutor va a ejecutar tu programa en una plataforma como Colab, así que conviene que el script funcione ahí con solo instalar `deltalake`.

Verificar que quedó todo bien

```
import time, pathlib, requests, pandas, deltalake, pyarrow
print("todo ok")
print("pandas", pandas.__version__, "| deltalake", deltalake.__version__)
```

OJO. Si al guardar el DataFrame te aparece `ImportError: Import pyarrow failed`, falta instalar `pyarrow`. Es la causa más frecuente de error en el paso de guardado.

OJO. En Colab los archivos viven en un disco temporal: cuando se cierra la sesión, la carpeta `datalake/` se borra. Para el TP no importa, porque el tutor corre el programa de cero y el código crea todo solo. Pero si querés conservar los datos, montá Google Drive.

Estructura de carpetas que va a crear el programa

```
datalake/
  bronze/
    artic/
      artwork_types/            <- tabla estatica (full, overwrite)
        _delta_log/             <- registro de transacciones de Delta
        part-0000....parquet    <- los datos
      artworks/                 <- tabla temporal (incremental, particionada)
        _delta_log/
        fecha=2026-10-26/
          hora=13/  part-....parquet
          hora=14/  part-....parquet
        fecha=2026-10-27/
          hora=09/  part-....parquet
```

No tenés que crear nada a mano: el programa arma todas esas carpetas solo, como pide el criterio 3a de evaluación.

4. La API del Art Institute of Chicago

Documentación oficial: https://api.artic.edu/docs/. Es una API pública, **no requiere clave ni registro**, y devuelve JSON.

Conceptos básicos de la API

- **URL base:** `https://api.artic.edu/api/v1`. Todos los endpoints cuelgan de ahí.
- Cada recurso (obras, artistas, lugares...) tiene un endpoint de listado (`/artworks`), uno de detalle (`/artworks/{id}`) y a veces uno de búsqueda (`/artworks/search`).
- Los nombres de campos van en minúscula y con guion bajo (`updated_at`, `artwork_type_title`).
- Todos los recursos tienen `id` y `title`.
- Los campos vacíos vienen como `null` (en Python, `None`), nunca como texto vacío.
- El listado de obras viene **ordenado por fecha de última actualización, de la más reciente a la más vieja**. Esa propiedad es la que hace posible el incremental.

Los parámetros que usamos

Parámetro	Qué hace	Detalle
`page`	Qué página pedir	Empieza en 1. Por defecto 1.
`limit`	Cuántos registros por página	Por defecto 12. **Máximo 100.** Con `limit=0` no trae datos pero sí el total (sirve para contar).
`fields`	Qué campos devolver	Lista separada por comas. Hace la respuesta más liviana y rápida. La documentación pide usarlo.
`ids`	Pedir varios registros por id	Ej: `ids=4,8,15`. No lo usamos acá.

Ejemplo de URL completa: `https://api.artic.edu/api/v1/artworks?page=2&limit=100&fields=id,title,updated_at`. Todo lo que va después del `?` son parámetros separados por `&`. Con `requests` no hace falta armarlos a mano: se pasan como diccionario en `params`.

Cómo es la respuesta

```
{
  "pagination": {
    "total": 44,           <- cuantos registros hay en total
    "limit": 3,            <- tamano de pagina pedido
    "offset": 0,
    "total_pages": 15,     <- cuantas paginas hay con ese limit
    "current_page": 1,
    "next_url": "https://api.artic.edu/api/v1/artwork-types?page=2&limit=3"
  },
  "data": [                <- LOS REGISTROS: esto es lo que queremos
    {"id": 48, "title": "Time Based Media", "aat_id": 300185191, ...},
    {"id": 34, "title": "Architectural Drawing", "aat_id": 300054197, ...}
  ],
  "info":   { "license_text": "...", "version": "1.13" },
  "config": { "iiif_url": "https://www.artic.edu/iiif/2", ... }
}
```

Lo que nos importa son dos cosas: `data` (la lista de registros) y `pagination["total_pages"]` (para saber cuándo dejar de pedir páginas). El resto (`info`, `config`) no lo usamos. Los valores exactos pueden variar con el tiempo porque el catálogo del museo cambia.

Límites y buenas prácticas que pide la API

Regla de la API	Qué hacemos nosotros
Usuarios anónimos: máximo **60 pedidos por minuto** por IP.	Esperamos 1 segundo entre pedidos (`time.sleep(1)`).
Si hacés scraping, un solo hilo y no más de 1 pedido por segundo.	Un solo hilo, secuencial, con la pausa anterior.
Usar `fields` para pedir solo lo necesario.	Lo usamos en `/artworks`.
`limit` no puede superar 100.	Usamos `limit=100`.
En endpoints de búsqueda no se puede pasar de 10.000 resultados.	No usamos búsqueda, usamos el listado.
Para volcados enormes (más de 10.000 registros), usar los data dumps.	Nuestro incremental trae pocos registros por corrida, así que la API alcanza.
Identificarse con el header `AIC-User-Agent`.	Lo incluimos (ver sección siguiente).

> **IDEA.** Esto es oro para el comentario de entrega: explicar que elegiste incremental porque la propia documentación desaconseja bajar el catálogo entero por la API, y que respetás el límite de pedidos, demuestra criterio profesional.

5. El header AIC-User-Agent

Qué es un header HTTP

Cada vez que tu programa (o tu navegador) pide algo a un servidor, además de la URL envía unos datos extra llamados **headers** o cabeceras: información sobre quién pide y cómo. Son pares nombre-valor. Por ejemplo:

Header	Para qué sirve
`User-Agent`	Identifica al programa que hace el pedido. Un navegador manda algo como "Mozilla/5.0 ... Chrome/..."; `requests` manda "python-requests/2.x".
`Accept`	Qué formato de respuesta prefiere el cliente (por ejemplo JSON).
`Authorization`	Credenciales o token, para APIs que lo exigen. Esta API no lo necesita.

Qué es AIC-User-Agent y por qué existe

El museo pide, **como cortesía**, que le agregues a cada pedido un header propio llamado `AIC-User-Agent` con el **nombre de tu proyecto y un mail de contacto**. El motivo: si algún programa genera una cantidad desproporcionada de tráfico, el equipo de ingeniería del museo puede identificarlo y escribirle al responsable para ayudarlo a optimizar sus consultas, en lugar de simplemente bloquearlo.

¿Por qué no usan el `User-Agent` común? Porque algunos navegadores **prohíben modificar** ese header desde código web. Entonces crearon uno propio. El prefijo `AIC-` es un "espacio de nombres" para reducir la chance de chocar con otras aplicaciones (es una práctica recomendada en el estándar RFC 6648).

Cómo se usa

Ejemplo de la documentación con `curl` (una herramienta de línea de comandos):

```
curl 'https://api.artic.edu/api/v1/artworks/24645' \
  --header 'AIC-User-Agent: aic-bash (engineering@artic.edu)'
```

Y en Python con `requests`, se arma un diccionario y se pasa en el parámetro `headers`:

```
HEADERS = {"AIC-User-Agent": "TP1-DataEngineering-UTN (tu_mail@ejemplo.com)"}

respuesta = requests.get(url, params=params, headers=HEADERS, timeout=30)
```

Preguntas frecuentes

- **¿Es obligatorio?** No. La API responde igual sin el header. Pero es lo correcto y lo recomienda la documentación.
- **¿Qué pongo?** Formato sugerido: `nombre-del-proyecto (tu_mail)`. Usá un mail real tuyo o de la facultad.
- **¿Es una clave de acceso?** No. No autentica ni da más permisos: es solo una identificación.
- **¿Cambia el límite de pedidos?** No. Sigue valiendo el límite de 60 por minuto por IP para usuarios anónimos. Si necesitás más, hay que escribirle a engineering@artic.edu.

> **TIP.** Definí `HEADERS` una sola vez, arriba de todo, como constante. Así lo reutilizás en todos los pedidos y si cambiás el mail lo cambiás en un solo lugar.

6. Las librerías en detalle

Para cada librería: qué hace, qué funciones usamos, qué opciones tienen y qué alternativas existen.

6.1 time (viene con Python)

Función	Qué hace
`time.sleep(segundos)`	Detiene el programa esa cantidad de segundos. Acepta decimales (`0.5`). La usamos para no pasar de 1 pedido por segundo.

6.2 pathlib (viene con Python)

Sirve para trabajar con rutas de archivos y carpetas como **objetos** en lugar de texto. Funciona igual en Windows, Mac y Linux.

Qué	Ejemplo	Explicación
Crear una ruta	`ruta = Path("datalake/bronze")`	Crea el objeto. No toca el disco todavía.
Unir partes	`Path("datalake") / "bronze" / "artic"`	El operador `/` une partes de la ruta con el separador correcto del sistema.
Crear carpetas	`ruta.mkdir(parents=True, exist_ok=True)`	`parents=True`: crea también las carpetas intermedias que falten. `exist_ok=True`: no da error si la carpeta ya existe.
Preguntar si existe	`ruta.exists()`	Devuelve `True` o `False`.
Convertir a texto	`str(ruta)`	`deltalake` pide la ruta como texto, por eso hacemos esta conversión.

> **OJO.** Sin `exist_ok=True`, la segunda vez que corrés el programa daría `FileExistsError`, porque la carpeta ya existe. Sin `parents=True`, daría error si no existe `datalake/bronze/` todavía.

6.3 requests

Es la librería estándar de Python para hablar con servidores web. La función principal es `requests.get`, que hace un pedido de tipo GET ("dame este recurso").

Argumentos de requests.get

Argumento	Qué es	Ejemplo
`url`	La dirección a pedir (primer argumento).	`"https://api.artic.edu/api/v1/artworks"`
`params`	Diccionario de parámetros de la URL. `requests` lo convierte a `?page=1&limit=100` y se ocupa de codificar caracteres especiales.	`{"page": 1, "limit": 100}`
`headers`	Diccionario de cabeceras.	`{"AIC-User-Agent": "..."}`
`timeout`	Segundos máximos de espera. **Siempre ponelo**: sin timeout, si el servidor no responde el programa puede quedarse colgado para siempre.	`timeout=30`

El objeto Response que devuelve

Atributo / método	Qué te da
`.status_code`	Código numérico del resultado (200, 404...). Ver tabla de abajo.
`.ok`	`True` si el código es menor a 400.
`.url`	La URL final usada, con los parámetros ya agregados. Muy útil para depurar.
`.text`	El contenido como texto crudo.
`.json()`	El contenido convertido a diccionario/lista de Python. Falla si la respuesta no es JSON válido.
`.raise_for_status()`	Si el código indica error (4xx o 5xx), lanza una excepción `HTTPError` y frena el programa. Si fue exitoso, no hace nada.

Códigos de estado que vas a ver

Código	Significa	Qué hacer
200	OK, todo bien.	Seguir.
404	No existe ese recurso o endpoint.	Revisar el nombre del endpoint (guiones, plural).
429	Demasiados pedidos.	Subir el `time.sleep` y reintentar más tarde.
500 / 502 / 503	Error del servidor.	No es tu culpa. Reintentar en unos minutos.

Excepciones principales

- `requests.exceptions.Timeout`: el servidor no respondió dentro del `timeout`.
- `requests.exceptions.ConnectionError`: no hay conexión a internet o no se pudo llegar al servidor.
- `requests.exceptions.HTTPError`: la lanza `raise_for_status()` cuando el código es de error.
- `requests.exceptions.RequestException`: clase "madre" de todas las anteriores; si querés atrapar cualquier fallo de red, usás esta.

6.4 pandas

Librería para trabajar con datos tabulares. Todo gira alrededor del `DataFrame`.

Función / atributo	Qué hace	Ejemplo
`pd.DataFrame(lista)`	Crea un DataFrame desde una lista de diccionarios. Cada diccionario es una fila.	`pd.DataFrame(registros)`
`df.head(n)`	Muestra las primeras n filas (5 por defecto).	`df.head()`
`df.shape`	Tupla (filas, columnas).	`(44, 7)`
`df.columns`	Nombres de las columnas.	`df.columns`
`df.dtypes`	Tipo de dato de cada columna.	`df.dtypes`
`df.empty`	`True` si el DataFrame no tiene filas.	`if df.empty:`
`df.copy()`	Hace una copia independiente, para modificar sin tocar el original.	`df = df.copy()`
`df.astype(str)`	Convierte todas las columnas a texto.	`df.astype(str)`

Función / atributo	Qué hace	Ejemplo
`df["col"]`	Selecciona una columna. También sirve para crear una: `df["nueva"] = ...`	`df["fecha"] = ...`
`pd.to_datetime(x, utc=True)`	Convierte texto a fecha/hora. Con `utc=True` normaliza todo a UTC y deja la fecha "con zona horaria".	`pd.to_datetime("2026-10-27T10:00:00-05:00", utc=True)`
`.dt.strftime(formato)`	Convierte una columna de fechas a texto con el formato indicado.	`df["updated_at"].dt.strftime("%Y-%m-%d")`
`pd.Timestamp.now(tz="UTC")`	La fecha y hora actual, en UTC.	
`pd.Timedelta(days=7)`	Una duración (7 días). Se puede restar a una fecha.	`ahora - pd.Timedelta(days=7)`
`.max()`	El valor más grande de una columna. Con fechas, la más reciente.	`serie.max()`

Formatos de fecha de strftime

Código	Significa	Ejemplo
`%Y`	Año con 4 dígitos	2026
`%m`	Mes con 2 dígitos	10
`%d`	Día con 2 dígitos	27
`%H`	Hora en formato 24h con 2 dígitos	14

`"%Y-%m-%d"` da `2026-10-27` y `"%H"` da `14`.

> **OJO. Por qué `utc=True`:** las fechas de la API vienen con una zona horaria (por ejemplo `-05:00`, hora de Chicago). Pandas no deja comparar una fecha "con zona" contra una "sin zona": da `TypeError: Cannot compare tz-naive and tz-aware`. Pasando todo a UTC se evita ese error y además se evita que una misma obra caiga en carpetas de fecha distintas según la hora local.

> **OJO. Por qué `astype(str)`:** si una columna es toda `null` en una corrida y tiene datos en otra, Pandas le asigna tipos distintos y Delta rechaza el cambio de esquema. Pasando todo a texto, el esquema es siempre el mismo. El costo: los vacíos quedan como el texto `"None"` o `"nan"`; limpiarlos es tarea del TP2. Para un bronze "crudo" es aceptable.

6.5 deltalake y pyarrow

`deltalake` (también llamada "delta-rs") es una implementación de Delta Lake para Python que **no necesita Spark**, por eso es ideal para un TP. `pyarrow` es la librería que le permite recibir los datos de Pandas; hay que tenerla instalada, aunque no la uses directamente.

write_deltalake(...)

Escribe un DataFrame como tabla Delta. Si la carpeta no existe, la crea.

Argumento	Qué es
`table_or_uri`	Ruta de la carpeta de la tabla, como texto. Primer argumento.
`data`	El DataFrame a guardar. Segundo argumento.
`mode`	Qué hacer si la tabla ya existe. Ver tabla siguiente.

Argumento	Qué es
`partition_by`	Lista de columnas por las que particionar, por ejemplo `["fecha", "hora"]`. Se define cuando se **crea** la tabla.
`schema_mode`	Permite cambiar el esquema al escribir: `"overwrite"` reemplaza el esquema en un overwrite, `"merge"` agrega columnas nuevas. No hace falta en nuestro código porque todo es texto.

Los modos de escritura (mode)

mode	Comportamiento	Cuándo usarlo
`"error"` (por defecto)	Si la tabla existe, **falla**.	Para protegerte de pisar algo sin querer.
`"overwrite"`	Reemplaza el contenido de la tabla por el nuevo.	**Extracción full** de datos estáticos.
`"append"`	Agrega las filas al final, sin revisar duplicados.	Si estás seguro de que lo que agregás es nuevo.
`"ignore"`	Si la tabla existe, no hace nada.	Carga inicial única.

> **TIP.** Con `overwrite` Delta **no borra el historial**: crea una versión nueva de la tabla y las anteriores quedan registradas en `_delta_log`. Por eso después de correr el full dos veces, `version()` sube, pero las filas siguen siendo 44.

DeltaTable

Es la clase para abrir y operar sobre una tabla que ya existe.

Qué	Código	Explicación
Preguntar si existe una tabla Delta	`DeltaTable.is_deltatable(ruta_texto)`	Devuelve `True`/`False`. Lo usamos para decidir entre crear la tabla o hacer merge.
Abrir una tabla	`tabla = DeltaTable(ruta_texto)`	Falla si la carpeta no es una tabla Delta.
Leer a Pandas	`tabla.to_pandas()`	Devuelve todo como DataFrame.
Leer solo algunas columnas	`tabla.to_pandas(columns=["updated_at"])`	Mucho más rápido: solo lee esa columna. Lo usamos para el watermark.
Versión actual	`tabla.version()`	Número de versión (0 la primera vez, sube con cada escritura).
Historial de cambios	`tabla.history()`	Lista de operaciones realizadas: qué se hizo y cuándo.
Viajar en el tiempo	`DeltaTable(ruta, version=0)`	Abre la tabla como estaba en esa versión.
Merge	`tabla.merge(...)`	Ver explicación abajo.

merge: insertar solo lo que no existe

```
(
    DeltaTable(str(ruta))
    .merge(
        source=df,                  # filas nuevas (llegan de la API)
        predicate="t.id = s.id AND t.updated_at = s.updated_at",
        source_alias="s",           # apodo del DataFrame nuevo
        target_alias="t",           # apodo de la tabla existente
    )
    .when_not_matched_insert_all()  # si no hay coincidencia, insertar
    .execute()                      # ejecutar la operacion
```

```
)
```

- `source` es lo nuevo (tu DataFrame) y `target` es lo que ya está en la tabla. Los alias `s` y `t` son apodos para escribir la condición.
- `predicate` es una condición en texto tipo SQL que define cuándo una fila nueva "ya existe". Acá: misma obra (`id`) **y** misma versión (`updated_at`).
- `when_not_matched_insert_all()`: las filas nuevas que no encontraron pareja en la tabla se insertan con todas sus columnas. Las que sí encontraron pareja se ignoran, y por eso no hay duplicados.
- `execute()` hace falta al final: hasta ahí solo estuviste describiendo la operación.
- Otras cláusulas existentes: `when_matched_update_all()` (actualizar las que coinciden) y `when_matched_delete()`. No las necesitamos.

> **IDEA.** Como el predicado incluye `updated_at`, si una obra se vuelve a modificar entra como **fila nueva**: la capa bronze guarda el historial de cambios de cada obra. Si prefirieras tener una sola fila por obra (siempre la última versión), usarías `predicate="t.id = s.id"` junto con `when_matched_update_all()`. Es una decisión de diseño que podés justificar en tu entrega.

7. Paso a paso

Construimos el programa pieza por pieza. Podés ir pegando cada bloque en tu archivo y probándolo antes de seguir. Al final del documento (sección 8) está todo junto.

PASO 1 Importar librerías y definir la configuración

Misión: preparar las herramientas y dejar en un solo lugar todos los valores que podrías querer cambiar.

```
import time
from pathlib import Path

import pandas as pd
import requests
from deltalake import DeltaTable, write_deltalake

BASE_URL = "https://api.artic.edu/api/v1"
HEADERS = {"AIC-User-Agent": "TP1-DataEngineering-UTN (tu_mail@ejemplo.com)"}

PAUSA_SEGUNDOS = 1          # la API pide no superar 1 pedido por segundo
LIMITE_POR_PAGINA = 100     # maximo permitido por la API
MAX_PAGINAS = 50            # tope de seguridad para el incremental
DIAS_PRIMERA_CARGA = 7      # si la tabla no existe, se traen los ultimos 7 dias

RUTA_TIPOS = Path("datalake/bronze/artic/artwork_types")
RUTA_OBRAS = Path("datalake/bronze/artic/artworks")

CAMPOS_OBRAS = (
    "id,title,artist_title,date_display,"
    "artwork_type_title,department_title,updated_at"
)
```

Explicación

- `import time` y `from pathlib import Path`: módulos de la librería estándar. Con `from ... import ...` traés solo la pieza que necesitás (`Path`) y la usás sin prefijo.
- `import pandas as pd` y `import requests`: librerías instaladas con pip. El `as pd` es un apodo que usa toda la comunidad para escribir menos.
- `from deltalake import DeltaTable, write_deltalake`: traés las dos herramientas de Delta Lake que usamos.
- Las constantes van en **MAYÚSCULAS**: es la convención de Python para valores que no cambian durante la ejecución.
- `MAX_PAGINAS = 50` evita que, por un error de lógica, el programa pida páginas sin fin. Con 100 obras por página son 5.000 obras como máximo por corrida.
- `CAMPOS_OBRAS` usa paréntesis y dos strings pegados: Python los concatena. Así la línea no queda kilométrica y se lee mejor.
- `Path("datalake/bronze/artic/artworks")` solo **crea el objeto ruta**: la carpeta recién se crea cuando llamemos a `mkdir`.

> **TIP.** Probalo: corré el archivo. Si no da ningún error, los imports y la instalación están bien.

PASO 2 Conocer la API antes de programar

Misión: ver con tus propios ojos qué devuelve la API, para entender qué vas a procesar.

Primero en el navegador: abrí `https://api.artic.edu/api/v1/artwork-types?limit=3`. Después, lo mismo desde Python (en un archivo aparte o en una celda de prueba):

```
import requests

url = "https://api.artic.edu/api/v1/artwork-types"
respuesta = requests.get(url, params={"limit": 3}, timeout=30)

print(respuesta.status_code)      # 200 significa OK
print(respuesta.url)              # la URL final, con ?limit=3 agregado
datos = respuesta.json()          # de texto JSON a diccionario de Python
print(datos.keys())               # claves: pagination, data, info, config
print(datos["pagination"])        # cuantas paginas hay, etc.
print(datos["data"][0])           # el primer registro
```

Qué deberías ver

- `200`.
- Una URL del estilo `https://api.artic.edu/api/v1/artwork-types?limit=3`.
- Las claves `pagination`, `data`, `info` y `config`.
- Un diccionario de `pagination` con `total` (alrededor de 44), `limit: 3` y `total_pages: 15`.
- Un registro con campos como `id`, `title`, `aat_id`, `updated_at`.

Probá cambiar `limit`, agregar `page=2` y mirar cómo cambia `datos["data"]`. Experimentar acá te ahorra mucho tiempo después.

PASO 3 La función para pedir datos

Misión: escribir una función reutilizable que haga un pedido a la API, cuide los límites y devuelva el JSON como diccionario.

```
def pedir_json(endpoint, params):
    """Hace un GET a la API y devuelve la respuesta como diccionario."""
    respuesta = requests.get(
        f"{BASE_URL}/{endpoint}",
        params=params,
        headers=HEADERS,
        timeout=30,
    )
    respuesta.raise_for_status()   # corta si la API devolvio un error (404, 500...)
    time.sleep(PAUSA_SEGUNDOS)     # respetar el limite de pedidos
    return respuesta.json()
```

Explicación línea por línea

- `def pedir_json(endpoint, params):` define la función. Recibe el nombre del endpoint (`"artworks"`) y un diccionario de parámetros.
- El texto entre triples comillas justo debajo del `def` es un **docstring**: documenta qué hace la función. La consigna evalúa que el código esté documentado, así que ponelo en todas.
- `f"{BASE_URL}/{endpoint}"` es un **f-string**: las llaves se reemplazan por el valor de las variables. Resultado: `https://api.artic.edu/api/v1/artworks`.
- `params`, `headers` y `timeout`: explicados en la sección 6.3.
- `raise_for_status()`: si algo salió mal, el programa se detiene **con un mensaje claro** en lugar de seguir con datos incorrectos.
- `time.sleep(PAUSA_SEGUNDOS)`: pausa de 1 segundo para cumplir el límite.
- `return respuesta.json()`: devuelve el diccionario. Esta función se usa en **todos** los pedidos del programa.

IDEA. Centralizar el pedido en una sola función es una buena práctica: si mañana querés agregar reintentos o logs, lo cambiás en un solo lugar y no en cada extracción.

Probala: `print(pedir_json("artwork-types", {"limit": 2}))`. Tiene que devolver el mismo diccionario que viste en el paso 2.

PASO 4 Extracción FULL de los tipos de obra

Misión: traer todos los registros de `/artwork-types`, recorriendo todas las páginas, y devolverlos como DataFrame.

```
def extraer_full(endpoint):
    """Recorre TODAS las paginas de un endpoint y devuelve un DataFrame."""
    registros = []
    pagina = 1
    while True:
        datos = pedir_json(endpoint, {"page": pagina, "limit": LIMITE_POR_PAGINA})
        registros.extend(datos["data"])
        if pagina >= datos["pagination"]["total_pages"]:
            break
        pagina += 1
    print(f"FULL {endpoint}: {len(registros)} registros")
    return pd.DataFrame(registros)
```

Explicación

- `registros = []`: lista vacía donde vamos juntando todo lo que llega.
- `while True:` es un bucle que se repite indefinidamente; lo vamos a cortar nosotros con `break`.
- `registros.extend(datos["data"])`: agrega **todos los elementos** de la lista nueva a `registros`. No confundir con `append`, que agregaría la lista entera como un único elemento y dejaría una lista de listas.
- `datos["pagination"]["total_pages"]`: cuántas páginas hay. Cuando la página actual llega a ese número, ya trajimos todo y hacemos `break`.
- `pagina += 1` avanza a la siguiente. Equivale a `pagina = pagina + 1`.
- `pd.DataFrame(registros)`: convierte la lista de diccionarios en tabla.

Probala:

```
df_tipos = extraer_full("artwork-types")
print(df_tipos.shape)    # unas 44 filas
print(df_tipos.head())
```

TIP. Con `limit=100` y 44 tipos, toda la tabla entra en una sola página: el bucle da una vuelta y corta. El código está escrito para funcionar igual con cualquier cantidad de páginas.

PASO 5 Guardar el FULL en Delta Lake

Misión: crear las carpetas si faltan y guardar el DataFrame pisando lo anterior.

```
def guardar_full(df, ruta):
    """Pisa la tabla completa (overwrite)."""
    ruta.mkdir(parents=True, exist_ok=True)
    write_deltalake(str(ruta), df.astype(str), mode="overwrite")
    print(f"Guardado FULL en {ruta} ({len(df)} filas)")
```

Explicación

- `ruta.mkdir(parents=True, exist_ok=True)`: asegura que las carpetas existan (criterio 3a de la consigna).

- `df.astype(str)`: pasa todo a texto, para que el esquema sea siempre el mismo entre corridas. Es la "leve transformación" que permite la consigna.
- `str(ruta)`: `deltalake` espera texto, no un objeto `Path`.
- `mode="overwrite"`: si la tabla ya existe, la reemplaza. Es lo que pide la consigna para datos que "se pisan".
- Los `print` dan visibilidad de lo que pasa; para un programa que corre sin pantalla son un recurso simple y útil.

Verificar que se guardó

```
guardar_full(df_tipos, RUTA_TIPOS)

tabla = DeltaTable(str(RUTA_TIPOS))
print(tabla.to_pandas().shape)    # mismas filas que df_tipos
print(tabla.version())            # 0 la primera vez
print(tabla.history())            # historial de operaciones
```

Corré `guardar_full` **dos veces** y mirá: `version()` sube (0, 1, 2...) pero las filas siguen siendo 44. Eso es `overwrite` funcionando: se reemplaza, no se acumula. Con esto ya tenés resuelta la mitad del TP.

PASO 6 El watermark: desde cuándo traer

Misión: averiguar la fecha de la última obra guardada para pedir solo lo posterior.

```
def obtener_watermark(ruta):
    """Devuelve la fecha de actualizacion mas reciente ya guardada."""
    if DeltaTable.is_deltatable(str(ruta)):
        guardado = DeltaTable(str(ruta)).to_pandas(columns=["updated_at"])
        if not guardado.empty:
            return pd.to_datetime(guardado["updated_at"], utc=True).max()
    return pd.Timestamp.now(tz="UTC") - pd.Timedelta(days=DIAS_PRIMERA_CARGA)
```

Explicación

- `DeltaTable.is_deltatable(...)` pregunta si ya existe una tabla Delta en esa ruta. **La primera vez no existe.**
- Si existe, leemos **solo la columna** `updated_at` (mucho más rápido que leer todo).
- `pd.to_datetime(..., utc=True)` convierte el texto a fechas reales, y `.max()` toma la más reciente. Ese es el watermark.
- `if not guardado.empty:` cubre el caso raro de una tabla que existe pero está vacía.
- Si la tabla no existe (primera corrida), el `return` final calcula "ahora menos 7 días". Es una **carga inicial acotada**: no tiene sentido bajar toda la historia del museo en el TP.

Probala: `print(obtener_watermark(RUTA_OBRAS))`. La primera vez tiene que imprimir una fecha de hace 7 días, con `+00:00` al final (eso indica UTC).

> **IDEA. Qué pasa si falla el programa a mitad de camino?** Como el watermark sale de lo que **efectivamente se guardó**, nunca se "adelanta" por error: la próxima corrida retoma desde el último dato bueno. Es una ventaja de leerlo desde la propia tabla.

PASO 7 Extracción INCREMENTAL de obras

Misión: traer solo las obras con `updated_at` posterior al watermark.

```
def extraer_incremental(endpoint, campos, desde):
    """Trae solo los registros con updated_at posterior a 'desde'."""
```

```
    registros = []
    for pagina in range(1, MAX_PAGINAS + 1):
        datos = pedir_json(
            endpoint,
            {"page": pagina, "limit": LIMITE_POR_PAGINA, "fields": campos},
        )
        lote = datos["data"]
        nuevos = [
            r for r in lote
            if pd.to_datetime(r["updated_at"], utc=True) > desde
        ]
        registros.extend(nuevos)
        # La API ordena de mas nuevo a mas viejo: si ya aparecio uno viejo, cortamos.
        if len(nuevos) < len(lote):
            break
    print(f"INCREMENTAL {endpoint}: {len(registros)} registros desde {desde}")
    return pd.DataFrame(registros)
```

La idea central

`/artworks` devuelve las obras ordenadas de la modificada más recientemente a la más antigua. Imaginá una pila de fichas ordenadas por fecha: empezás por arriba y sacás fichas **hasta que encontrás una que ya tenías**. Todo lo que está más abajo es todavía más viejo, así que no hace falta seguir.

```
Pagina 1:  [nueva][nueva][nueva]...[nueva]   -> todas nuevas: seguir con la pagina 2
Pagina 2:  [nueva][nueva][vieja][vieja]...   -> aparecio una vieja: CORTAR (break)
                           ^
                           watermark (lo ultimo que ya estaba guardado)
```

Explicación

- `for pagina in range(1, MAX_PAGINAS + 1)`: recorre las páginas 1 a 50. `range(1, 51)` genera 1, 2, ..., 50 (el final no se incluye, por eso el `+ 1`).
- `"fields": campos` pide solo las columnas listadas (obligatorio incluir `updated_at`, porque la usamos para filtrar).
- `lote = datos["data"]`: las obras de esta página.
- El bloque con corchetes es una **list comprehension**: construye una lista nueva filtrando. Es equivalente a este bucle:

```
nuevos = []
for r in lote:
    if pd.to_datetime(r["updated_at"], utc=True) > desde:
        nuevos.append(r)
```

- `if len(nuevos) < len(lote): break`: si en la página había obras viejas (hay menos nuevas que el total), ya llegamos al límite y cortamos.
- Si **todas** las de la página son nuevas, el bucle sigue con la página siguiente.
- Si la tabla ya está al día, la primera página tiene 0 o muy pocas nuevas y el programa termina en **un solo pedido**: eso es lo que hace eficiente al incremental.

> **OJO.** Esta lógica depende de que la API mantenga el orden por actualización descendente, que es lo que indica su documentación para el listado. Si algún día cambiara el orden, el corte temprano dejaría de ser válido; en ese caso habría que recorrer todo o usar el endpoint de búsqueda con un filtro por fecha.

Probala (con un watermark de ejemplo):

```
desde = obtener_watermark(RUTA_OBRAS)
df_obras = extraer_incremental("artworks", CAMPOS_OBRAS, desde)
print(df_obras.shape)    # unas centenas o miles de obras de los ultimos 7 dias
print(df_obras.head())
```

Si te trae decenas de miles, algo anda mal con el filtro. Si trae cero, puede ser que la API no haya actualizado nada en los últimos 7 días: probá con `DIAS_PRIMERA_CARGA = 30`.

PASO 8 Preparar fecha y hora para particionar

Misión: crear las columnas `fecha` y `hora` y dejar el DataFrame listo para guardar.

```
def preparar_particiones(df):
    """Agrega las columnas fecha y hora (para particionar) y pasa todo a texto."""
    if df.empty:
        return df
    df = df.copy()
    df["updated_at"] = pd.to_datetime(df["updated_at"], utc=True)
    df["fecha"] = df["updated_at"].dt.strftime("%Y-%m-%d")
    df["hora"] = df["updated_at"].dt.strftime("%H")
    return df.astype(str)
```

Explicación

- `if df.empty: return df`: si no hay obras nuevas, no hay nada que preparar. Sin esta guarda, la línea de `updated_at` daría `KeyError` porque un DataFrame vacío no tiene columnas.
- `df.copy()`: trabajamos sobre una copia para no modificar el DataFrame original sin querer.
- Convertimos `updated_at` a fecha real en UTC para poder extraer partes de ella.
- `.dt.strftime("%Y-%m-%d")` da `2026-10-27` y `.dt.strftime("%H")` da `14`. Son las columnas de partición que pide la consigna.
- `return df.astype(str)`: todo a texto. `updated_at` queda como `2026-10-27 14:05:33+00:00`, formato que `pd.to_datetime` vuelve a leer sin problema en el paso 6.

Probala: `print(preparar_particiones(df_obras).head())`. Tenés que ver las columnas `fecha` y `hora` al final.

PASO 9 Guardar el INCREMENTAL sin duplicar

Misión: crear la tabla particionada la primera vez, y en las siguientes insertar solo lo que no existe.

```
def guardar_incremental(df, ruta):
    """Inserta solo las filas nuevas, sin duplicar."""
    if df.empty:
        print("No hay registros nuevos para guardar")
        return
    ruta.mkdir(parents=True, exist_ok=True)
    if not DeltaTable.is_deltatable(str(ruta)):
        write_deltalake(str(ruta), df, partition_by=["fecha", "hora"])
    else:
        (
            DeltaTable(str(ruta))
            .merge(
                source=df,
                predicate="t.id = s.id AND t.updated_at = s.updated_at",
                source_alias="s",
                target_alias="t",
            )
            .when_not_matched_insert_all()
            .execute()
        )
    print(f"Guardado INCREMENTAL en {ruta} ({len(df)} filas procesadas)")
```

Los tres casos

Situación	Qué hace el código
El DataFrame está vacío (no hay obras nuevas)	Avisa con un mensaje y sale con `return`. No toca la tabla.

Situación	Qué hace el código
La tabla **todavía no existe** (primera corrida)	`write_deltalake(..., partition_by=["fecha", "hora"])` la crea, ya particionada.
La tabla **ya existe**	`merge`: inserta solo las filas cuya combinación `id` + `updated_at` no esté ya en la tabla.

Por qué merge y no append

Si usaras `mode="append"` y el script se corriera dos veces con el mismo watermark (por ejemplo, porque se cortó antes de terminar), las mismas obras quedarían duplicadas. Con `merge`, la segunda corrida detecta que ya están y las ignora. Esa es la **idempotencia** de la que hablamos en los conceptos.

> **OJO.** `partition_by` solo se indica **al crear** la tabla. Al hacer merge, Delta respeta las particiones que ya tiene: las filas nuevas se acomodan solas en la carpeta `fecha=.../hora=...` que les corresponde.

Probala:

```
df_obras = preparar_particiones(df_obras)
guardar_incremental(df_obras, RUTA_OBRAS)
```

Después revisá en tu explorador de archivos: tienen que aparecer carpetas como `datalake/bronze/artic/artworks/fecha=2026-10-27/hora=14/`.

PASO 10 Unir todo en main() y la prueba de fuego

Misión: armar el programa completo y comprobar que el incremental no duplica.

```
def main():
    # 1) Datos estaticos: full + overwrite
    df_tipos = extraer_full("artwork-types")
    guardar_full(df_tipos, RUTA_TIPOS)

    # 2) Datos temporales: incremental + merge particionado
    desde = obtener_watermark(RUTA_OBRAS)
    df_obras = extraer_incremental("artworks", CAMPOS_OBRAS, desde)
    df_obras = preparar_particiones(df_obras)
    guardar_incremental(df_obras, RUTA_OBRAS)


if __name__ == "__main__":
    main()
```

Explicación

- `main()` es la función principal: lee como una receta de alto nivel (extraer tipos, guardarlos, calcular watermark, extraer obras, preparar, guardar). Ese orden narrativo es lo que se evalúa en "calidad del código".
- `if __name__ == "__main__":` hace que `main()` se ejecute **solo si corrés este archivo directamente**, y no si alguien lo importa desde otro. Es la forma estándar de Python.
- En un notebook (Colab) alcanza con llamar `main()` en una celda.

La prueba de fuego

```
from deltalake import DeltaTable

def contar():
    return len(DeltaTable(str(RUTA_OBRAS)).to_pandas())

main()
print("filas despues de la 1ra corrida:", contar())
main()
```

```
print("filas despues de la 2da corrida:", contar())
```

Tiene que dar **el mismo número** (o muy poco más, si el museo actualizó alguna obra en el medio). La segunda corrida debería mostrar `INCREMENTAL artworks: 0 registros` o muy pocos, y el mensaje "No hay registros nuevos para guardar". Si el número se duplicó, revisá el `predicate` del merge.

PASO 11 Ordenar y entregar

Misión: dejar el código prolijo y escribir el comentario con tus decisiones.

- Revisá que **cada función tenga docstring** y que los nombres sean descriptivos (`extraer_full`, no `f1`).
- Dejá los valores configurables como constantes arriba (ya está hecho).
- Agregá al inicio del archivo un comentario general que diga qué hace el programa, qué API usa y qué endpoints.
- Renombrá el archivo con **tu nombre y apellido +** `_TP1`, por ejemplo `JuanPerez_TP1.py` o `JuanPerez_TP1.ipynb`.
- Probalo **desde cero** en Colab: borrá la carpeta `datalake`, reiniciá el entorno y corré todo. Tiene que funcionar sin intervención.
- Escribí el comentario de entrega (ver sección 11).

8. Script completo

Es el mismo código de los pasos, todo junto. Lo probé con datos simulados: la primera corrida cargó las obras, la segunda insertó solo las nuevas y la tercera no insertó nada. Los nombres de campos salen de la documentación de la API; confirmalo corriendo el paso 2 con tu conexión.

```
"""
TP1 - Data Engineering (UTN BA)
Extraccion desde la API del Art Institute of Chicago y guardado en Delta Lake.

- /artwork-types : datos estaticos   -> extraccion FULL    -> overwrite
- /artworks      : datos temporales  -> extraccion INCREMENTAL -> merge particionado
"""

import time
from pathlib import Path

import pandas as pd
import requests
from deltalake import DeltaTable, write_deltalake

# ---------------------------------------------------------------------
# Configuracion
# ---------------------------------------------------------------------
BASE_URL = "https://api.artic.edu/api/v1"
HEADERS = {"AIC-User-Agent": "TP1-DataEngineering-UTN (tu_mail@ejemplo.com)"}

PAUSA_SEGUNDOS = 1          # la API pide no superar 1 pedido por segundo
LIMITE_POR_PAGINA = 100     # maximo permitido por la API
MAX_PAGINAS = 50            # tope de seguridad para el incremental
DIAS_PRIMERA_CARGA = 7      # si la tabla no existe, se traen los ultimos 7 dias

RUTA_TIPOS = Path("datalake/bronze/artic/artwork_types")
RUTA_OBRAS = Path("datalake/bronze/artic/artworks")

CAMPOS_OBRAS = (
    "id,title,artist_title,date_display,"
    "artwork_type_title,department_title,updated_at"
)


# ---------------------------------------------------------------------
# Extraccion
# ---------------------------------------------------------------------
def pedir_json(endpoint, params):
    """Hace un GET a la API y devuelve la respuesta como diccionario."""
    respuesta = requests.get(
        f"{BASE_URL}/{endpoint}",
        params=params,
        headers=HEADERS,
        timeout=30,
    )
    respuesta.raise_for_status()   # corta si la API devolvio un error (404, 500...)
    time.sleep(PAUSA_SEGUNDOS)     # respetar el limite de pedidos
    return respuesta.json()


def extraer_full(endpoint):
    """Recorre TODAS las paginas de un endpoint y devuelve un DataFrame."""
    registros = []
    pagina = 1
    while True:
        datos = pedir_json(endpoint, {"page": pagina, "limit": LIMITE_POR_PAGINA})
        registros.extend(datos["data"])
        if pagina >= datos["pagination"]["total_pages"]:
            break
        pagina += 1
    print(f"FULL {endpoint}: {len(registros)} registros")
    return pd.DataFrame(registros)


def obtener_watermark(ruta):
```

```
    """Devuelve la fecha de actualizacion mas reciente ya guardada."""
    if DeltaTable.is_deltatable(str(ruta)):
        guardado = DeltaTable(str(ruta)).to_pandas(columns=["updated_at"])
        if not guardado.empty:
            return pd.to_datetime(guardado["updated_at"], utc=True).max()
    return pd.Timestamp.now(tz="UTC") - pd.Timedelta(days=DIAS_PRIMERA_CARGA)


def extraer_incremental(endpoint, campos, desde):
    """Trae solo los registros con updated_at posterior a 'desde'."""
    registros = []
    for pagina in range(1, MAX_PAGINAS + 1):
        datos = pedir_json(
            endpoint,
            {"page": pagina, "limit": LIMITE_POR_PAGINA, "fields": campos},
        )
        lote = datos["data"]
        nuevos = [
            r for r in lote
            if pd.to_datetime(r["updated_at"], utc=True) > desde
        ]
        registros.extend(nuevos)
        # La API ordena de mas nuevo a mas viejo: si ya aparecio uno viejo, cortamos.
        if len(nuevos) < len(lote):
            break
    print(f"INCREMENTAL {endpoint}: {len(registros)} registros desde {desde}")
    return pd.DataFrame(registros)


# ---------------------------------------------------------------------
# Transformaciones leves
# ---------------------------------------------------------------------
def preparar_particiones(df):
    """Agrega las columnas fecha y hora (para particionar) y pasa todo a texto."""
    if df.empty:
        return df
    df = df.copy()
    df["updated_at"] = pd.to_datetime(df["updated_at"], utc=True)
    df["fecha"] = df["updated_at"].dt.strftime("%Y-%m-%d")
    df["hora"] = df["updated_at"].dt.strftime("%H")
    return df.astype(str)


# ---------------------------------------------------------------------
# Almacenamiento en Delta Lake
# ---------------------------------------------------------------------
def guardar_full(df, ruta):
    """Pisa la tabla completa (overwrite)."""
    ruta.mkdir(parents=True, exist_ok=True)
    write_deltalake(str(ruta), df.astype(str), mode="overwrite")
    print(f"Guardado FULL en {ruta} ({len(df)} filas)")


def guardar_incremental(df, ruta):
    """Inserta solo las filas nuevas, sin duplicar."""
    if df.empty:
        print("No hay registros nuevos para guardar")
        return
    ruta.mkdir(parents=True, exist_ok=True)
    if not DeltaTable.is_deltatable(str(ruta)):
        write_deltalake(str(ruta), df, partition_by=["fecha", "hora"])
    else:
        (
            DeltaTable(str(ruta))
            .merge(
                source=df,
                predicate="t.id = s.id AND t.updated_at = s.updated_at",
                source_alias="s",
                target_alias="t",
            )
            .when_not_matched_insert_all()
            .execute()
        )
    print(f"Guardado INCREMENTAL en {ruta} ({len(df)} filas procesadas)")
```

```
# ----------------------------------------------------------------------
# Programa principal
# ----------------------------------------------------------------------
def main():
    # 1) Datos estaticos: full + overwrite
    df_tipos = extraer_full("artwork-types")
    guardar_full(df_tipos, RUTA_TIPOS)

    # 2) Datos temporales: incremental + merge particionado
    desde = obtener_watermark(RUTA_OBRAS)
    df_obras = extraer_incremental("artworks", CAMPOS_OBRAS, desde)
    df_obras = preparar_particiones(df_obras)
    guardar_incremental(df_obras, RUTA_OBRAS)


if __name__ == "__main__":
    main()
```

9. Cómo verificar que está bien

Ver la estructura de carpetas

En Colab (o cualquier terminal Linux/Mac):

```
!find datalake -maxdepth 5 -type d
```

En Windows, abrí la carpeta `datalake` en el explorador. Tienen que existir `_delta_log` en cada tabla y las subcarpetas `fecha=.../hora=...` en `artworks`.

Inspeccionar las tablas

```
tipos = DeltaTable(str(RUTA_TIPOS))
obras = DeltaTable(str(RUTA_OBRAS))

print(tipos.to_pandas().shape)          # unas 44 filas
print(obras.to_pandas().shape)          # filas acumuladas de obras
print(obras.to_pandas().head())
print(obras.version())                  # sube con cada merge que inserto algo
print(obras.history())                  # que operacion se hizo en cada version
```

Chequeos que conviene hacer

Chequeo	Cómo	Resultado esperado
¿Se pisan los tipos?	Correr `main()` dos veces; mirar `tipos.to_pandas().shape`	Las filas no cambian; `version()` sube.
¿Se evita duplicar obras?	Contar filas, correr de nuevo, contar de nuevo	El mismo número, o poco más.
¿Hay particiones?	Listar carpetas de `artworks`	Carpetas `fecha=.../hora=...`.
¿Se crean las carpetas solas?	Borrar `datalake/` y correr	Todo se vuelve a crear sin error.
¿Filas repetidas?	`df.duplicated(subset=["id", "updated_at"]).sum()`	0

10. Errores comunes

Qué ves	Por qué pasa	Cómo se arregla
`ModuleNotFoundError: No module named 'deltalake'`	Falta instalar la librería (o la instalaste en otro Python distinto del que corre el código).	`pip install deltalake pyarrow`. En Colab, con `!` adelante y reiniciando el entorno si hace falta.
`ImportError: Import pyarrow failed`	Pandas necesita `pyarrow` para pasarle los datos a Delta.	`pip install pyarrow`.
`HTTPError: 429 Too Many Requests`	Pediste más de 60 veces por minuto.	Subí `PAUSA_SEGUNDOS` a 2 y esperá un minuto antes de reintentar.
`HTTPError: 404 Not Found`	El endpoint está mal escrito.	Revisá guiones y plural: `artwork-types`, no `artwork_types`.
`requests.exceptions.Timeout` o `ConnectionError`	Sin internet, o el servidor tardó demasiado.	Revisá la conexión y reintentá.
`TypeError: Cannot compare tz-naive and tz-aware datetime`	Comparás una fecha con zona horaria contra una sin zona.	Usá `utc=True` en **todos** los `pd.to_datetime`.
`KeyError: 'updated_at'`	El DataFrame está vacío o pediste `fields` sin `updated_at`.	Verificá `df.empty` antes de procesar e incluí `updated_at` en `CAMPOS_OBRAS`.
`TableNotFoundError` o error al abrir la tabla	Intentás abrir con `DeltaTable(...)` una carpeta que no existe o no es Delta.	Antes de abrir, preguntá con `DeltaTable.is_deltatable(...)`.
Error de `schema mismatch` al escribir	Los tipos de las columnas cambiaron entre corridas.	Pasá todo a texto con `astype(str)` (ya está en el código).
`FileExistsError`	Hiciste `mkdir` sin `exist_ok=True`.	Usá `mkdir(parents=True, exist_ok=True)`.
Se duplican filas en la tabla de obras	Usaste `append` en lugar de `merge`, o el predicado no compara bien.	Revisá `predicate="t.id = s.id AND t.updated_at = s.updated_at"`.
Desapareció `datalake/` en Colab	Se reinició o cerró la sesión; el disco es temporal.	Es normal. El código la recrea al correr de nuevo.

11. Entrega

Checklist contra los criterios de evaluación

Criterio de la consigna	Dónde se cumple
1a Código bien estructurado y con buenas prácticas	Funciones chicas con una sola responsabilidad, constantes arriba, `main()`, `if __name__ == "__main__"`.
1b Funcionamiento documentado de forma clara y concisa	Docstring en cada función, comentario general al inicio y comentarios solo donde aclaran.
2a Justificar la técnica de extracción	Texto de entrega (abajo): incremental en obras, full en tipos.
2b Extraer de la fuente con la técnica elegida	`extraer_full` y `extraer_incremental`, usando `requests` sobre 2 endpoints.
3a Asegurar que existan los directorios, o crearlos	`ruta.mkdir(parents=True, exist_ok=True)` antes de cada escritura.
3b Pisar datos o insertar nuevos según corresponda	`overwrite` en la tabla estática; `merge` con inserción de nuevas filas en la temporal.
Particionar por fecha y hora en incremental	`partition_by=["fecha", "hora"]`.
DataFrames de Pandas y formato Delta Lake	`pd.DataFrame(...)` y `write_deltalake(...)`.
Nombre de archivo	`NombreApellido_TP1.py` o `.ipynb`.
Ejecutable por el tutor desde Colab	Solo requiere `!pip install deltalake` al principio.

Texto modelo para el comentario de la entrega

La consigna pide subir un mensaje con los argumentos de tus decisiones. Podés adaptar este texto con tus palabras:

> **TIP.** Reescribilo con tus propias palabras y ajustalo a lo que realmente hiciste. Tiene que reflejar que entendiste cada decisión, porque el tutor puede preguntarte.

```
Seleccion de la fuente
Elegi la API publica del Art Institute of Chicago porque no requiere
autenticacion, esta bien documentada y ofrece datos con distinta naturaleza:
un endpoint con datos que cambian en el tiempo (/artworks, con el campo
updated_at) y otro de metadatos estaticos (/artwork-types).

Tecnica de extraccion
- /artworks: extraccion INCREMENTAL. Son mas de 100.000 registros y la propia
  documentacion desaconseja bajarlos por la API. El listado viene ordenado por
  ultima actualizacion, asi que en cada corrida traigo solo las obras con
  updated_at posterior al maximo ya guardado (watermark) y corto la
  paginacion cuando aparece una anterior.
- /artwork-types: extraccion FULL. Es un catalogo de unas 44 filas que casi no
  cambia; es mas simple y seguro bajarlo completo y pisar la tabla.

Almacenamiento en Delta Lake (capa bronze)
- artwork_types: modo overwrite, sin particionar por ser una tabla chica.
- artworks: particionada por fecha y hora de updated_at. Para insertar uso
  MERGE con la condicion (id, updated_at) en lugar de append, de modo que el
  proceso sea idempotente: si se ejecuta dos veces no duplica datos, y si una
  obra se modifica queda registrada como nueva version.
- Los directorios se crean automaticamente si no existen.

Buenas practicas
Respeto el limite de la API (pausa de 1 segundo entre pedidos), pido solo los
campos necesarios con el parametro fields, uso limit=100 (maximo permitido),
timeout en cada pedido e identifico el programa con el header AIC-User-Agent.
```

```
Transformaciones
Solo agrego las columnas fecha y hora para particionar y convierto las
columnas a texto para mantener un esquema estable. La limpieza queda para
la proxima entrega.
```

Ideas opcionales para destacar

- Agregar **reintentos** en `pedir_json`: si falla, esperar unos segundos y volver a intentar hasta 3 veces antes de rendirse.
- Reemplazar los `print` por el módulo `logging` (con niveles INFO y ERROR).
- Sumar un tercer endpoint estático (por ejemplo `/category-terms`) para mostrar que la función `extraer_full` es reutilizable.
- Guardar en la tabla una columna `fecha_extraccion` con el momento en que se bajó cada dato (es una práctica común en bronze).
- Escribir un `README` corto con las instrucciones para ejecutar.

> **IDEA.** Cuando tengas tu versión armada, pegámela y la revisamos juntos: criterios de la consigna, nombres, docstrings y casos borde. Y cuando llegue el TP2 (limpieza y procesamiento), esta tabla bronze va a ser tu punto de partida.